

STRSsim Code Guide

End-to-end walkthrough of the cohesin loop-extrusion simulation codebase

2026

Contents

CODE_GUIDE: What Every File in This Repo Actually Does	1
STRS Project: Quick-Start and Codebase Walkthrough	1
The Snakemake pipeline files	5
Reading order	7
Project layout	7
1. Configuration: <code>configs/parameters.py</code>	8
2. Hi-C data preparation	9
3. CTCF annotation	11
4. The LEF dynamics engine	12
5. Running simulations	13
6. Post-simulation analysis	15
Cluster submit scripts	20
Conventions you'll see everywhere	24
Common pitfalls	24

CODE_GUIDE: What Every File in This Repo Actually Does

This is a plain-language walkthrough of the whole codebase. It is written for people who don't read Python and want to understand, end-to-end, what runs, what each script produces, and how the files feed into each other.

If you just want to run the pipeline, see [README.md](#). Come back here when you need to know *why* a step exists, what an output file contains, or where a specific variable is set.

STRS Project: Quick-Start and Codebase Walkthrough

This section covers the STRS-specific configuration — HEK293T cells, hg38, hyperosmotic stress. The rest of this guide was written for an earlier version of the codebase (mESC/Sox2/mm10) and still describes the mechanics correctly; just substitute STRS-specific names where noted.

What this repo does

STRSsim is a physics-based simulation framework for testing mechanistic hypotheses about how hyperosmotic stress (sorbitol treatment) reorganizes 3D chromatin in HEK293T cells. The core question: **can we explain the stress-induced changes in Hi-C contact maps purely from what we can measure — changes in CTCF binding and cohesin dynamics — without invoking anything we can't observe?**

The simulation models cohesin loop extrusion: cohesin rings load onto chromatin and extrude DNA into loops, stalling when they encounter CTCF sites oriented the right way. Where cohesins stall and how long they live determines the 3D contact map. By varying which CTCF sites are present (control vs. sorbitol CUT&Tag peaks) and cohesin dynamics parameters, the simulation tests five specific hypotheses:

Condition	Cohesin	CTCF sites	Tests
control_ctcf-control	Control	Control	Does the model reproduce untreated Hi-C?
control_ctcf-sorbitol	Control	Sorbitol	Does CTCF loss alone explain sorbitol Hi-C?
weak_ctcf-sorbitol	Control	Sorbitol (0.5× stall)	Does weaker stalling at remaining CTCF sites help?
sorbitol_promoter-stall	Sorbitol	Promoter anchors	Can promoter-anchored stalling reproduce the new loops?
sorbitol_promoter-stall_Song	Sorbitol (2× processivity)	Promoter anchors	Promoter stalling + longer cohesin runs

Data flow

```
STRS CUT&Tag peaks (.narrowPeak)
  ↓ extract_ctcf_sites_hg38.py    (add strand orientation via FIMO)
Oriented CTCF BED files
  ↓ run_simulation.py + polychrom  (cohesin loop extrusion + 3D polymer)
Simulated contact maps
  ↓ run_analysis_all.py
Comparison figures ↔ real Hi-C (.mcool, from .hic via hic2cool)
```

Setup: run this once

```
cd /work/users/j/p/jpflores/projects/STRSsim
bash setup_data.sh
```

setup_data.sh handles everything end-to-end:

1. **Step 1** — creates the two conda environments (`cohesin_sim` and `ctcf_extraction`) if they don't already exist.
2. **Steps 1–3** — symlinks your `.hic` Hi-C files from the STRS project and converts them to `.mcool` at 1 kb resolution (takes ~10–15 min; run from an interactive node: `srun --mem=32G --cpus-per-task=4 --pty bash`).
3. **Step 4** — runs `extract_ctcf_sites_hg38.py` via `conda run -n ctfc_extraction` for all 4 loci $\times$ 2 conditions (8 BED files total). Uses FIMO to find the JASPAR CTCF motif in each peak and assign strand orientation.
4. **Step 5** — symlinks the oriented BEDs to the paths `configs/parameters.py` expects.
5. **Step 6** — validates all BED files before any simulation runs.

After setup, pick a locus in `configs/parameters.py`:

```
ACTIVE_LOCUS = "chr1_fig1"    # chr1:64,210,000-66,170,000  (JAK1/AK4/LEPR)
# other options: "chr4_fig1", "chr6_fig1", "chr16 Sox8"
```

Then launch:

```
sbatch SimPipe.sh
```

The central config: `configs/parameters.py`

This one file is the source of truth for everything. At the top, `ACTIVE_LOCUS` selects the genomic region; from that, `CHROM`, `REGION_START`, `REGION_END`, and `N_MONOMERS` (2,000 — one per kb) are derived automatically.

CTCF auto-loading. When any script imports `configs.parameters`, it immediately tries to open `data/ctcf_oriented_hg38_STRS_control_<locus>.bed` and the equivalent sorbitol file. It parses each line into (`monomer_position`, `orientation`) where +1 means a forward-strand motif and -1 means reverse. If the files don't exist yet, it falls back to placeholder dummy sites and prints a warning.

Cohesin parameters. Five parameter sets are defined, each a dict with `lifetime` (mean steps before cohesin falls off), `separation` (spacing between cohesins), `ctcf_capture` (probability of stalling per CTCF encounter), and `ctcf_release` (escape probability per step while stalled). Values come from Gabriele et al. 2022 (mESC measurements used as the human baseline).

Tiling. Rather than simulating 2,000 monomers, the simulation runs 70,000 monomers arranged as 28 back-to-back copies of the locus (each tile = 2,000 monomers + 250 padding on each side). All 28 tiles are independent, so each GPU run produces $\sim 28\times$ the contact statistics at no extra physics cost. Contact maps are folded back down to $2,000\times 2,000$ for comparison.

The CTCF extraction: `scripts/extract_ctcf_sites_hg38.py`

Your `.narrowPeak` file has peak coordinates but no strand information. Orientation matters because a + CTCF site blocks a cohesin arm moving leftward, while a - site blocks one moving

rightward — so a convergent pair (+ on the left, - on the right) creates a stably anchored loop, while a divergent pair does not.

This script:

1. Takes each peak's summit from the CUT&Tag narrowPeak file.
2. Extracts the DNA sequence at each peak with `bedtools getfasta`.
3. Runs FIMO to scan for the JASPAR MA0139.1 CTCF motif.
4. Uses the motif's strand as the site orientation.
5. Writes a BED6 file where column 6 is + or -.

The 1D engine: `scripts/lef_dynamics.py`

Before any 3D physics, loop extrusion is modelled as a purely 1D stochastic process. `LEFSimulator` runs this:

- Each cohesin is a pair of arms that step away from each other along the chromatin fiber, one lattice site per simulation tick.
- An arm stalls when it hits a CTCF site with the blocking orientation, with probability `ctcf_capture`. It escapes with probability `ctcf_release` per step.
- Arms also stall when they collide with another cohesin (steric exclusion).
- Each cohesin lives for a geometrically distributed number of steps with mean `lifetime`, then unbinds and is replaced at a random position.

The 3D simulation: `scripts/run_simulation.py`

This is where polychrom (OpenMM on GPU) runs. For each block:

1. `LEFSimulator.step()` advances all cohesins one tick on the 1D lattice.
2. Cohesin arm positions feed into polychrom as harmonic spring bonds between the two bridged monomers.
3. OpenMM integrates the polymer under those bonds for 250 MD steps.
4. Every 10 blocks, all monomer coordinates are saved and pairs within 3.0 monomer-units of each other are recorded as contacts.

The analysis: `analysis/`

- `contact_maps.py` — accumulates saved conformations into a 2D contact frequency matrix using `scipy.spatial.KDTree` for fast distance lookups. This is the simulated analogue of a Hi-C contact map.
- `ps_curve.py` — computes $P(s)$: contact probability as a function of genomic separation. The shape of this curve is the primary comparison metric against real Hi-C.
- `experimental_compare.py` — loads your `.mcool` file via `cooler` and extracts the same locus at 1 kb resolution for direct comparison.
- `msd_two_point.py` — tracks mean-squared displacement of specific monomer pairs over time (the in-silico analogue of live-cell imaging).

The cluster pipeline: `SimPipe.sh` + `Snakemake`

The pipeline is a Snakemake DAG (`workflows/STRSsim.snakefile`) launched by a single `sbatch SimPipe.sh`. Snakemake submits each rule as its own SLURM job through

snakemake-executor-plugin-slurm:

```
convert_hic (*2, CPU)
  → extract_ctcf (*2, CPU, FIMO)
    → validate_ctcf (*1, CPU)
      → simulate_shard (*60, GPU - 5 cond × 3 rep × 4 shards)
        → merge_shards (*15, CPU - 5 cond × 3 rep)
          → analyze_all (*5, CPU - one per condition)
```

To dry-run (see all jobs without submitting): `bash SimPipe.sh --dry-run`. If a run fails and leaves a lock: `bash unlock.sh`.

See [The Snakemake pipeline files](#) below for a full description of every new file.

The Snakemake pipeline files

These files were added to replace the legacy `cluster/submit_pipeline.sh` bash-script chain with a proper Snakemake DAG. Each downstream rule only runs after its inputs exist — no manual dependency bookkeeping required.

SimPipe.sh — sbatch entry point

Submit with `sbatch SimPipe.sh`. It does three things:

1. Sets up a local Python venv in `.snakemake_venv/` (first run only; uses `module load python/3.12.4`).
2. `pip` installs Snakemake 8.27.1 + `snakemake-executor-plugin-slurm`.
3. Calls `snakemake --profile profiles/slurm --configfile config/SimConfig.yaml --jobs 100 --rerun-incomplete --latency-wait 500`.

The venv is reused on subsequent runs. `bash SimPipe.sh --dry-run` prints every SLURM command without submitting.

unlock.sh — release lock after failure

Snakemake writes a `.snakemake/locks/` directory while running. If the job dies without cleanup, the lock persists and the next run refuses to start. `bash unlock.sh` runs `snakemake --unlock` against the workflow to clear it.

workflows/STRSsim.snakefile — the DAG

Six rules, in order:

Rule	SLURM resources	What it does
<code>convert_hic</code>	4 CPU, 32 GB	<code>hic2cool</code> convert <code>.hic</code> → <code>.mcool</code> at 1 kb
<code>extract_ctcf</code>	2 CPU, 16 GB	<code>extract_ctcf_sites_hg38.py</code> via <code>ctcf_extraction</code> env
<code>validate_ctcf</code>	1 CPU, 4 GB	<code>validate_ctcf.py</code> — sanity-checks all BED files

Rule	SLURM resources	What it does
<code>simulate_shard</code>	1 GPU, 4 CPU, 64 GB	<code>run_simulation_shard.py</code> — 60 jobs total ($5 \times 3 \times 4$)
<code>merge_shards</code>	4 CPU, 32 GB	<code>merge_shards.py</code> — concatenates shard HDF5 streams
<code>analyze_all</code>	4 CPU, 32 GB	<code>run_analysis_all.py</code> — contact maps, P(s), MSD, figures

Wildcard `{condition}` matches the 5 condition names from `SIMULATION_CONDITIONS` in `configs/parameters.py`. Wildcards `{rep}` and `{shard}` expand over `range(n_replicates)` and `range(n_shards)` from `config/SimConfig.yaml`.

Locus, conditions, and cohesin parameters all come from `configs/parameters.py` at parse time — there is no duplication.

Snakemake tracks completion via sentinel `.done` files in `results/.sentinels/`. Actual simulation data lives in `results/polychrom_3d/` with the flat naming convention the scripts already expect.

`profiles/slurm/config.yaml` — SLURM executor defaults

Tells `snakemake-executor-plugin-slurm` which account and partition to use for all rules by default. Per-rule overrides (GPU partition, GPU count, memory, walltime) are set in each rule's `resources:` block in the Snakefile.

Key defaults:

- `slurm_account:` `rc_dphansti_pi`
- `slurm_partition:` `general` (GPU rules override to `gpu`)
- `runtime:` 4320 (72 h fallback for long-running jobs)
- Log files: `logs_slurm/{rule}.{wildcards}.{jobid}.out/err`

`config/SimConfig.yaml` — path configuration

Edit **once** before the first run. Key fields:

Field	What to change
<code>hic_control</code> / <code>hic_sorbitol</code>	Absolute paths to your STRS <code>.hic</code> maps
<code>genome</code>	Path to hg38 FASTA (default: Longleaf shared reference)
<code>samplesheet</code>	Path to <code>SimSamplesheet.txt</code>
<code>n_replicates</code>	Independent simulation replicates (default: 3)
<code>n_shards</code>	GPU shards per replicate (default: 4, gives $4 \times$ frames per GPU run)

All other parameters (locus, cohesin params, tiling) are read from `configs/parameters.py` and do NOT need to be duplicated here.

SimSamplesheet.txt — CTCF input file registry

Tab-separated, two columns: CTCF_Type and CTCF_Peaks_Path.

CTCF_Type	CTCF_Peaks_Path
control	/path/to/STRS_HEK293...CTCF_cont_0h_peaks.narrowPeak
sorbitol	/path/to/STRS_HEK293...CTCF_sorbitol_1h_peaks.narrowPeak

The `extract_ctcf` rule reads this file via `pandas.read_table()` at runtime. If the STRS project moves, update the paths here (not in the Snakefile).

Reading order

Everything here is organised by pipeline stage, top-down:

1. [The Snakemake pipeline files](#) — **start here for the STRS pipeline**
2. [Project layout](#) — what each folder is for
3. [Configuration](#) — the single file that controls every simulation
4. [Hi-C data preparation](#) — turning experimental data into comparison matrices
5. [CTCF annotation](#) — turning CUT&Tag peaks into oriented barrier lists
6. [The LEF dynamics engine](#) — the loop-extrusion simulator
7. [Running simulations](#) — the one-shot, batch, and multi-GPU entry points
8. [Post-simulation analysis](#) — contact maps, P(s), APA, MSD, calibration
9. [Cluster submit scripts](#) — legacy sbatch wrappers (superseded by SimPipe.sh)
10. [Conventions you'll see everywhere](#) — monomers, tiling, output directory names
11. [Common pitfalls](#) — what to check when something looks wrong

Every script section follows the same format:

- **File** — path in the repo
- **Purpose** — one-sentence summary
- **Inputs** — command-line flags and files read
- **Outputs** — files written, with exact paths
- **How to run it** — example command
- **What it does, step by step** — plain-language walkthrough

Project layout

cohesin_sim/	
configs/	- Python file with every tunable parameter
data/	- experimental inputs (Hi-C matrices, CTCF beds, genomes)
scripts/	- Python + shell scripts you run by hand or via SLURM
process_hic/	- bash + Python to turn FASTQ / valid-pairs into mcool
ctcf_annotator/	- standalone module for CTCF motif orientation
analysis/	- Python modules called by scripts/run_analysis_all.py
cluster/	- SLURM submit scripts (<code>`sbatch <file>.sh`</code>)
results/	- where simulations and analyses write to (git-ignored)
logs/	- SLURM stdout/stderr (create with <code>`mkdir -p logs`</code>)

The `old/` directory holds the previous version of each script for reference; nothing there is wired into the current pipeline. Ignore it.

1. Configuration: `configs/parameters.py`

Purpose. This single file is the source of truth for everything a simulation needs to know: which genomic region to model, how long the polymer chain is, how many cohesins to place on it, how fast they unbind, where the CTCF roadblocks sit, which experimental datasets to compare against, and which eight “conditions” to run. If you edit one number here, every simulation and analysis picks it up automatically.

Key groups of settings:

- **Genomic locus** (`CHROM`, `REGION_START`, `REGION_END`, `RESOLUTION`, `N_MONOMERS`). The window is chr3:34,000,000–36,000,000 on mm10, at 1 kb per monomer → 2,000 monomers for the base locus.
- **Polymer physics** (`POLYMER` dict). Bond lengths, bending stiffness (`angle_force`), confinement density, integration tolerance. You rarely touch these unless you’re re-tuning the physics.
- **LEF polymer bonds** (`SMC_BOND`). How stiff the cohesin “ring” is in 3D — how strongly it pulls the two flanks of the loop together.
- **CTCF tracks** (`CTCF_BED_MESC`, `CTCF_BED_NEURON`). Relative paths to BED files under `data/`. At import time, `load_ctcf_from_bed()` reads them and fills `CTCF_SITES_MESC` / `CTCF_SITES_NEURON` — two NumPy arrays of monomer indices with +1 / -1 orientation labels. The shipped defaults are:
 - **mESC** → `data/ctcf_oriented_mm10_mESC_Bruce4_chr3_34000000_36000000.bed` (ENCODE Bruce4 mESC, ENCFF508CKL, mm10; 26 oriented sites in the 2 Mb window, 13 + / 13 -).
 - **neuron** → `data/ctcf_oriented_mm10_GSE96107_CN_chr3_34000000_36000000.bed` (Bonev 2017 cortical neuron ChIP-seq, GSE96107, mm10; 16 oriented sites, 8 + / 8 -).
 - 11 sites are conserved between the two tracks with **exact monomer index + matching orientation**. These are what the MSD probes anchor on (see §[MSD pair policy](#)).
- **Cohesin parameter sets** (`ALL_PARAM_SETS`). One dict per cell-type hypothesis (mESC baseline + six CN variants) with five numbers each: **lifetime** (mean steps a cohesin lives before falling off), **separation** (average distance between cohesins, used to compute how many to place), **ctcf_capture** (probability of stalling on contact), **ctcf_release** (probability per step a stalled cohesin escapes), plus a short **name**.
- **Condition list** (`SIMULATION_CONDITIONS`). The 8 rows of the matrix in the README — each pairs one parameter set with one CTCF track. This is what the SLURM scripts iterate over.
- **Tiling** (`TILING`). Instead of simulating the 2 Mb locus on its own (which gives noisy statistics), we copy it 28 times end-to-end into one long polymer so each simulation produces ~28× more data per wall-clock second. `TILING["chrom_size"]` is the total length (~70,000 monomers); `tile_size = 2500`, `padding = 250`, so every tile is 2,000 useful monomers + 250 monomers of padding on each side to suppress boundary artefacts.
- **Run parameters** (`SIM_RUN`). How many total simulation blocks, how many MD steps per block, how many LEF steps per block, how often to save a conformation, how long to warm up before saving.
- **MSD pair selection** (`MSD_PROBE`, `MSD_FIT`, plus the helper `get_msd_pairs(cell_type)`).

Which two monomers to track the mean-squared distance between for the dynamics analysis, and how to fit the resulting curve. The default policy (conserved CTCF anchors flanking Sox2, identical indices in both cell types) is described in detail in [§MSD pair policy](#).

- **Calibration** (CALIBRATION). Whether to anchor physical units (“how many nanometres is a monomer?”, “how many seconds is a frame?”) against experimental Hi-C P(s) decay, against experimental MSD, or skip.

Two helper functions you’ll see called elsewhere:

- `get_ctcf_arrays(cell_type)` — returns (positions, orientations) for the untiled 2 Mb locus.
- `get_tiled_ctcf_arrays(cell_type)` — same but repeated across the tiled chromosome (what the simulations actually use).
- `get_condition(name)` — look up one row of SIMULATION_CONDITIONS by name; used by every SLURM script.

2. Hi-C data preparation

The goal of this stage is to end up with two NumPy matrices on disk — `data/hic_mESC_Sox2.npy` and `data/hic_CN_Sox2.npy` — each a 2,000×2,000 balanced Hi-C matrix of the Sox2 locus at 1 kb resolution, ready to be compared to simulated contact maps.

You have two routes: fast (chr3-only, ~1 hour) or full (raw FASTQ → mcool, 1–2 days).

2a. Fast path: `scripts/download_bonev_hic.sh`

Purpose. Download the Bonev 2017 Hi-C **valid pairs** for chr3 (already aligned and quality-controlled by lldelisle), index them, load them into a balanced `.cool`, zoomify to a multi-resolution `.mcool`.

Inputs. No arguments (downloads both ES and CN). Pass ES or CN as the first argument to do just one.

Outputs. Under `data/mcool/`: `-ES_chr3_raw.1kb.cool / CN_chr3_raw.1kb.cool` — raw counts at 1 kb. `-ES_chr3.1kb.cool / CN_chr3.1kb.cool` — balanced, single-resolution. `-ES_chr3.mcool / CN_chr3.mcool` — multi-resolution (1, 5, 10, 25, 50, 100 kb), balanced.

How to run it. Three modes:

```
# Local, both cell types in sequence
```

```
bash scripts/download_bonev_hic.sh
```

```
# Local, both in parallel in the same shell
```

```
bash scripts/download_bonev_hic.sh ES &
```

```
bash scripts/download_bonev_hic.sh CN &
```

```
wait
```

```
# Cluster (~30-60 min each on 40 cores)
```

```
sbatch --export=CELLTYPE=ES scripts/sbatch_bonev_hic.sbatch
```

```
sbatch --export=CELLTYPE=CN scripts/sbatch_bonev_hic.sbatch
```

What it does, step by step.

1. Makes sure `data/mcool/` and `data/valid_pairs/` exist.
2. Writes out mm10 chromosome sizes (hardcoded list) if they aren't already there.
3. For each cell type, `wgets` the matching `validPairs.csort.txt.gz` from the ldelisle FTP mirror of GSE161259, streaming to a `.part` file so an interrupted download doesn't leave something that looks complete.
4. `tabix-indexes` the `valid-pairs` file so cooler can load it in chunks.
5. Deletes any partial balanced/mcool outputs from a previous crashed run (belt and braces — a header-only HDF5 would otherwise be treated as “done” by the `if [ ! -f ... ]` guards).
6. `cooler cload tabix` loads pair counts into a raw 1 kb cool file. We pass "`<chromsizes>:<binsize>`" directly because `cooler makebins` was renamed to `cooler binnify` in cooler 0.10, and the old spelling silently produces a 0-byte bins file that only blows up much later.
7. `cooler zoomify --balance` upscales to a multi-resolution `.mcool` and runs ICE balancing (`--cis-only`) at every level in one pass.
8. Extracts the 1 kb balanced level into a standalone `.cool` with `cooler cp` for people who want it for pyGenomeTracks.

Parallelism knobs. `NPROC` (default = `SLURM_CPUS_PER_TASK` or 8) is passed to `cooler cload --nproc` and as `--max-split` so chr3 is carved into as many chunks as there are workers. Tied together they keep every core busy — this is the fix for the old “feels stuck” behaviour when only two workers had anything to do.

2b. Full path: `scripts/process_hic/00_download_fastq.sh` → `01_align_and_parse.sh` → `02_make_mcool.sh`

Purpose. Rebuild Bonev's Hi-C from raw reads using a modern aligner (`bwa-mem2`) and pair parser (`pairtools`). Use this for the final paper-quality analysis; it takes ~1–2 days wall-clock and ~1 TB of disk.

Pipeline.

1. `00_download_fastq.sh` — `fasterq-dump` every SRR run for mESC and CN from SRA into `data/fastq/`.
2. `01_align_and_parse.sh` — for each `<SRR>_{1,2}.fastq`: align with `bwa-mem2 mem -SP5M`, classify pair types with `pairtools parse`, sort with `pairtools sort`, deduplicate with `pairtools dedup`. Output: `data/pairs/<SRR>.pairs.gz`.
3. `02_make_mcool.sh` — merge all pairs per cell type, bin into a 1 kb `.cool` with `cooler cload pairix`, zoomify + ICE-balance to `.mcool` at 1/5/10/25/50/100 kb.

On a cluster, chain them with `--dependency=afterok:<jobid>` or submit the wrapper `cluster/submit_hic_pipeline.sh` which handles the chain.

2c. Locus extraction: `scripts/process_hic/03_extract_sox2.py`

Purpose. Pull the 2 Mb Sox2 window out of the whole-chromosome `.mcool` and save it as a plain NumPy `.npy` matrix so the analysis code doesn't have to know anything about cooler.

Inputs: - `--mcool-es PATH` — mESC `.mcool` (e.g. `data/mcool/ES_chr3.mcool`). - `--mcool-cn PATH` — neuron `.mcool`. - `--output DIR` — where to write `.npy` files. Default `data`.

Outputs (written to `--output`).

File	Contents
hic_mESC_Sox2.npy / hic_CN_Sox2.npy	Balanced 2,000×2,000 Hi-C matrix of the locus — this is what every comparison reads.
hic_mESC_Sox2_raw.npy / hic_CN_Sox2_raw.npy	Raw (unbalanced) counts, kept for reference.
hic_mESC_Sox2_OE.npy / hic_CN_Sox2_OE.npy	Observed/Expected matrices — removes the distance-decay background so TADs and loops stand out.
hic_mESC_Sox2_expected.npy / hic_CN_Sox2_expected.npy	The expected-vs-distance curve that was divided out to make OE.
hic_<ct>_ps_curve.npz	Contact probability vs distance (P(s)) from the experimental matrix.
insulation_<ct>.npy	Insulation score along the locus (one value per monomer).
insulation_<ct>_w<window>.npz	Insulation at several window sizes for robustness.
compartment_<ct>.npy	First eigenvector of the corrected matrix (compartment call, A vs B).

2d. Quicklook: scripts/plot_hic_quicklook.py

Purpose. Before kicking off a multi-hour analysis, eyeball the Hi-C maps. This script draws a heatmap with a CTCF arrow strip, a gene track (fetched live from UCSC), and Mb-labelled axes, and also writes a split-triangle comparison with both cell types on one square.

Outputs. Under data/: - hic_<CT>_Sox2_quicklook.png / .svg — one panel per cell type. - hic_dual_Sox2_quicklook.png / .svg — upper-right triangle = first cell type, lower-left = second, one shared log colour scale so redder really means “more contact”.

Every SVG has `svg.fonttype='none'` so labels stay editable `<text>` elements in Illustrator, with fonts pinned to Arial.

3. CTCF annotation

3a. scripts/ctcf_annotator/ — the standalone tool

Purpose. Given a CTCF ChIP-seq peak BED and a genome FASTA, find the best-scoring CTCF motif inside each peak with FIMO, use the motif’s strand as the site orientation (+1 = forward, -1 = reverse), and write an oriented BED.

This is a self-contained CLI with its own README inside the folder.

Entry points: - `python -m scripts.ctcf_annotator <args>` — the CLI. - `scripts/ctcf_annotator/gui.py` — optional Tk GUI (skipped silently without `tkinter`).

Key modules: - `core.py` — the actual annotation pipeline (FIMO wrapper + BED writer). - `cli.py` — argparse interface. - `registry.py` — hardcoded URLs for common ENCODE datasets so you can do `--dataset ENCODE:mESC_Bruce4_CTCF` instead of pasting a URL.

Inputs (typical): `--peaks <bed>, --genome <fa>, --output <bed>, --motif <meme-file>` (default: JASPAR MA0139.1 CTCF at `data/motifs/MA0139.1_CTCF.meme`).

Outputs: a BED file of oriented CTCF sites — one row per peak that contained a usable motif, with strand in column 6.

3b. `scripts/extract_ctcf_sites.py` / `extract_ctcf_sites_hg38.py`

Purpose. Convenience wrappers that download an ENCODE CTCF peak bed, call the annotator, and produce `ctcf_oriented_*.bed` files ready to be loaded by `configs/parameters.py`.

`extract_ctcf_sites.py` targets mm10; `extract_ctcf_sites_hg38.py` targets hg38. They duplicate a lot of code and should ideally be merged into one with an `--assembly` flag (future work).

3c. `scripts/validate_ctcf.py`

Purpose. Pre-flight sanity check. Before running any simulations, load the two CTCF BEDs, convert genomic coordinates into monomer indices exactly the way the simulator will, and print a small report.

How to run it.

```
# Defaults (reads paths from configs/parameters.py):
python scripts/validate_ctcf.py

# Explicit file paths:
python scripts/validate_ctcf.py \
    --mesc data/ctcf_oriented_mm10_GSE96107_ES_chr3_34000000_36000000.bed \
    --neuron data/ctcf_oriented_mm10_GSE96107_CN_chr3_34000000_36000000.bed
```

What it checks.

1. Each BED parses without error and has strand info.
2. Every site falls inside the simulation window.
3. Coordinate $\rightarrow$ monomer-index conversion is monotone and loss-free.
4. The +1 / -1 orientation array the simulator sees matches the strand column in the BED.
5. Prints counts per cell type (e.g. “mESC: 84 sites, neuron: 16 sites”) and highlights sites shared / lost / gained between conditions.

Exit status. Non-zero if any check fails, so you can use it as a gate in a dependency chain.

3d. `scripts/export_conserved_ctcf_bed.py`

Purpose. Intersect the mESC and neuron oriented BEDs, keep only sites present in both (with matching orientation), optionally allowing 1-monomer drift. Writes `data/ctcf_oriented_CONSERVED_mESC_CN_ch` which you can load into a simulation to test the “CTCF didn’t move, only cohesin changed” scenario.

4. The LEF dynamics engine

`scripts/lef_dynamics.py`

Purpose. A lightweight 1D Python simulator for loop-extruding factors (cohesins). No 3D physics — just: where are the cohesins on the lattice, where are they stalled, where do they jump when they unbind.

Class: LEFSimulator. Constructor arguments:

- `N` — lattice length in monomers.
- `n_lefs` — how many cohesins to place.

- `lifetime` — mean number of steps a cohesin stays bound before unbinding (`unbind probability per step = 1/lifetime`).
- `ctcf_positions`, `ctcf_orientations` — from `get_ctcf_arrays`.
- `ctcf_capture` — probability of stalling when a cohesin arm moves onto a CTCF site with the blocking orientation.
- `ctcf_release` — probability per step that a stalled cohesin lets go.
- `rng_seed` — for reproducibility.

What each `step()` call does.

1. Rebuild the occupancy grid from every cohesin's left / right arm positions (so collisions are up-to-date if arms moved last step).
2. For each cohesin (in a random permutation, so no directional bias):
 - Age by one step.
 - With probability `1/lifetime`, `unbind` → clear occupancy, call `_place_lef()` to re-seed at a random free pair of positions.
 - Left arm (moves leftward): if the next position is free and not a blocking CTCF site (or we rolled `> ctcf_capture`), move there. If it is blocking and we rolled `< ctcf_capture`, flag as stalled.
 - Right arm (moves rightward): mirror logic.
 - If already stalled on CTCF, roll `< ctcf_release` to escape.

Public methods you'll see used.

- `get_bonds()` — list of (`left`, `right`) tuples of current cohesin-bridged monomer pairs. Fed to polychrom as harmonic SMC bonds.
- `get_bond_arrays()` — same but as two parallel NumPy arrays.
- `get_loop_sizes()` — current loop lengths (in monomers).
- `run(n_steps)` — convenience: step `n_steps` times, return the list of per-step bond lists.

Output variables you should know. After a `step()`, these arrays reflect the current state: `left_pos[i]`, `right_pos[i]`, `ages[i]`, `left_stalled[i]`, `right_stalled[i]`, `occupied[pos]`.

5. Running simulations

5a. `scripts/run_simulation.py` — single-run entry point

Purpose. Run one condition × one replicate → write a simulation directory under `results/`. This is what every SLURM array job calls.

Inputs (arguments).

Flag	Meaning
<code>--condition NAME</code>	Look up one row of <code>SIMULATION_CONDITIONS</code> . Sets both cohesin params and CTCF track. Preferred.
<code>--params NAME --ctcf-type {mESC,neuron}</code>	Legacy: pair any <code>ALL_PARAM_SETS</code> entry with any CTCF track.
<code>--replicate N</code>	Integer replicate index (drives the RNG seed <code>42 + 1000·N</code>).
<code>--gpu I</code>	CUDA device index for the 3D polymer (ignored in <code>lef_only</code>).

Flag	Meaning
<code>--output DIR</code>	Where to write the run directory (default <code>results/</code>).
<code>--engine</code>	Which engine to use. <code>auto</code> tries polychrom, falls back to
<code>{auto,polychrom,openmm,lef_only}</code>	OpenMM, then LEF-only. <code>lef_only</code> skips the 3D polymer even if polychrom is installed.

Output directory. Always written as `<output>/<params_name>_ctcf-<ctcf_type>_rep<replicate>/`. For example: `results/polychrom_3d/CN_long_residency_ctcf-neuron_rep0/`.

Files the run writes.

Regardless of engine: - `params.json` — every parameter used, including the CTCF type and tiling block, so you can reproduce the run from the directory alone.

Polychrom / OpenMM path: - `blocks_*.h5` (polychrom) or `conformations.h5` (OpenMM) — 3D monomer coordinates at every saved frame.

LEF-only path (either `--engine lef_only` or no polychrom/openmm installed): - `lef_contact_map.npy` — 2D integer matrix: how many frames had a bond between monomer *i* and monomer *j*. - `lef_trajectories.h5` — per-frame bond lists + loop sizes.

What it does, step by step.

1. Parse the condition → resolve to `(params_dict, ctf_type)`.
2. Build the tiled chromosome: `N = TILING["chrom_size"]`, fetch tiled CTCF arrays, decide how many cohesins to place (`N / params.separation`).
3. Instantiate `LEFSimulator` and step it for `warmup_blocks` (equilibration — throw away).
4. **Pre-compute** all LEF bond states for every block in advance. The 1D dynamics are independent of the 3D polymer, so doing them first means the polychrom loop just replays bonds from a list.
5. Collect every unique bond pair ever observed and register them all into one `harmonic_bonds` force with the non-active ones at `k=0`. Every block, flip the `k` of the bonds that became active / inactive — this is $\sim 10\times$ faster than adding/removing bonds.
6. `sim.local_energy_minimization()` relaxes the starting configuration.
7. Main loop: update bond toggles → `sim.do_block(md_steps)` → repeat for `total_blocks`.
8. polychrom's `HDF5Reporter` writes conformations every `save_every` blocks.

If polychrom isn't installed or `--engine lef_only`, step 4 still happens but steps 5–8 are replaced by an accumulating 2D contact map (how often does each (`left`, `right`) pair appear across saved frames).

5b. `scripts/run_batch.py` — batch runner

Purpose. Run many conditions $\times$ many replicates without SLURM (useful for local dev / testing), then immediately run the built-in analysis.

How to run it.

```
# Everything, LEF-only, 3 replicates:
python scripts/run_batch.py --engine lef_only
```

Only two conditions:

```
python scripts/run_batch.py --conditions mESC_ctrl CN_long_residency_neuron_ctcf
```

Analyse existing results without re-running:

```
python scripts/run_batch.py --analyze-only \  
    --exp-es data/hic_mESC_Sox2.npy --exp-cn data/hic_CN_Sox2.npy
```

What it does. Loops over SIMULATION_CONDITIONS (or the ones you listed), calls `run_simulation_polychrom` for each, then for every run directory it found: loads the contact map, computes $P(s)$, computes an insulation score, and compares to mESC + neuron Hi-C. Prints a summary table at the end ordered by which condition each row compares best to.

5c. scripts/run_simulation_shard.py + scripts/merge_shards.py

Purpose. Scale simulations to $4 \times$ V100 nodes. Instead of one long simulation per replicate, each GPU runs a *shard* with a unique RNG seed, and all shards are then merged into a single conformation file.

run_simulation_shard.py arguments (called once per GPU):

Flag	Meaning
--condition NAME	Same as <code>run_simulation.py</code> .
--replicate N	Replicate index.
--shard-index I	Which shard in this launch ($0 \dots n_shards - 1$).
--n-shards K	Total shards for this replicate.
--gpu I	CUDA device.
--output DIR	Output base (default <code>results/</code>).
--shard-index-offset M	Add M to every shard index so this run doesn't clash with shards already on disk.

Shard output goes to `<output>/<cond>_rep<r>_shard<i>/`. The total blocks are divided evenly across shards, so merging them gives the same number of frames as one long run would have, but wall-clock is divided by `n_shards`.

merge_shards.py. Walks `<cond>_rep<r>_shard*`, concatenates their HDF5 files into `<cond>_rep<r>/blocks_*.h5` (the same layout the analysis expects), and writes a `merge_metadata.json` with the shard IDs merged. Arguments: `--condition`, `--replicate`, `--n-shards`, `--results-dir`, `--cleanup` (delete shard dirs after merge), `--all` (merge every condition / replicate in one go).

6. Post-simulation analysis

6a. The orchestrator: scripts/run_analysis_all.py

Purpose. One command that walks every `<cond>_rep<N>` directory under `results/polychrom_3d/`, runs the full suite of analyses, pools across replicates per condition, and writes a cross-condition overlay.

How to run it (directly or via `cluster/submit_analysis_cpu.sh`):

```
python scripts/run_analysis_all.py \  

```

```
--results-dir results/polychrom_3d \
--mcool-mesc data/mcool/ES_chr3.mcool \
--mcool-neuron data/mcool/CN_chr3.mcool
```

Selected flags.

Flag	What it does
--results-dir DIR	Where to find the merged simulation directories. Required.
--output-dir DIR	Where to write pooled outputs. Default: <results-dir>/../analysis.
--condition NAME	Only analyse one condition — useful for iteration.
--n-jobs N	Parallel workers for contact detection inside each directory.
--no-parallel	Walk directories sequentially (debugging).
--skip-existing	Skip directories that already have analysis/sim_contact_map.npy.
--reuse-heavy	Re-use cached contact maps + MSD, redo only plots/APA/pooling. Use after editing plotting code.
--no-pool / --no-apa /	Skip specific stages.
--no-ctcf-overlay /	
--no-msd /	
--no-polymer-dynamics	
--mcool-mesc PATH /	Experimental Hi-C for comparison (takes precedence over legacy .npy).
--mcool-neuron PATH	
--ctcf-bed-mesc PATH /	Override the CTCF BEDs used for the overlay figure.
--ctcf-bed-neuron PATH	
--elements-bed-mesc PATH /	Optional BEDs of enhancers/promoters to draw on the 1D track.
--elements-bed-neuron PATH	
--calibrate-with {hic,msd,none}	How to map simulation units → physical units.

Per-replicate outputs (written to <sim_dir>/analysis/):

File	What it is
sim_contact_map.npy	2D contact matrix from 3D conformations.
sim_ps_curve.npz	Contact probability vs genomic distance.
sim_insulation.npy	Insulation score along the locus.
ps_metrics.json	Power-law fit of $P(s)$ (slope, intercept, R^2).
apa_convergent.png + apa_loops_quant.json	Aggregate Peak Analysis at convergent CTCF pairs.
contact_map_with_ctcf.png	Log heatmap with CTCF arrow strips + gene track.
ctcf_sites_relative.bed	CTCF sites in monomer-index coordinates.
comparison_metrics.json	Pearson + stratum-adjusted correlations with experimental Hi-C.
sim_vs_exp_map.png	Side-by-side heatmap of sim vs experimental Hi-C.
msd_<label>.npz + .json	Two-point MSD curves for each monomer pair.

File	What it is
<code>rg_timecourse.npz</code>	Radius of gyration over time.
<code>loop_fractions.json</code>	Fraction of time each pair was in contact.
<code>calibration.json</code>	Physical-unit mapping: <code>nm/monomer</code> , <code>sec/frame</code> , anchor used.

Pooled-per-condition outputs (written to `<output-dir>`):

Same family of files, prefixed with the condition name and the total number of blocks, e.g. `CN_long_residency_ctcf-neuron_5000blk_pooled_contact_map.npy`.

6b. analysis/contact_maps.py

Called from `run_analysis_all.py`. Given a list of conformations, builds a contact matrix by finding every pair of monomers within `SIM_RUN["contact_radius"]` using `scipy.spatial.cKDTree`. Supports memory-efficient tile-based extraction so you never materialise a full (N, N) intermediate.

Also has:

- `load_conformations_h5(sim_dir)` — read polychrom `blocks_*.h5` or OpenMM `conformations.h5`.
- `load_lef_contact_map(sim_dir)` — read the `.npy` written by the LEF-only path.
- `compute_ps_curve(contact_map, resolution)` — average contacts at each genomic separation.
- `compute_insulation_score(contact_map)` — classic Crane/Dekker-style insulation.
- `compare_contact_maps(sim, exp)` — Pearson + stratum-adjusted correlations on matched-size matrices.

6c. analysis/ps_curve.py

Power-law fitting for $P(s)$. Returns `(slope, intercept, r_squared)` with the fit range hardcoded to match Hansen-lab defaults (~ 10 kb – 1 Mb).

6d. analysis/absolute_quant.py

Aggregate Peak Analysis: pile every convergent CTCF pair on top of each other and report the average contact enrichment. This is the go-to summary statistic for loop strength.

6e. analysis/ctcf_plotting.py

Drawing the contact-map figures with CTCF arrows overlaid. Same visual grammar as `plot_hic_quicklook.py` for the experimental panel.

6f. analysis/experimental_compare.py

Matches a simulated contact map to an experimental Hi-C matrix (resolution-aware; handles both `.npy` and `.mcool`). Produces `sim_vs_exp_map.png` and numeric comparison metrics.

6g. analysis/msd_two_point.py + polymer_dynamics.py

msd_two_point.py. Computes the mean-squared displacement of the *vector* between two monomers as a function of lag time. This is the in-silico analogue of two-colour live-cell imaging (Gabriele 2022 on Fbn2, Mach 2022 *Nat Genet*). Fits a power law to extract the sub-diffusive exponent and the plateau.

MSD pair policy: conserved sites flanking Sox2 Which two monomers we track is not arbitrary. Picking a pair that is a strong CTCF-stalled anchor in one cell type but a weak / absent anchor in the other would produce an MSD difference that reflects “different anchor” rather than “different cohesin dynamics”. To avoid that confound, the default policy in `MSD_PROBE["auto"]` (in `configs/parameters.py`) enforces three rules simultaneously:

1. **Conservation** (`require_conserved = True`). The candidate set is the intersection of the mESC and neuron oriented BEDs: a site is kept only if it appears in *both* tracks at the same monomer index and the same orientation. With the shipped BEDs this yields **11 conserved sites** (strict, tolerance 0 monomers — bump `conserved_tol_monomers` to 1–2 if you suspect off-by-one noise in the site caller). Because the indices match across cell types, the MSD curves from mESC and neuron simulations are comparable pixel-for-pixel — no renormalisation needed.
2. **Proximity to Sox2** (`anchor_bp, prefer_flanking`). Pairs are ranked by whether they flank the Sox2 midpoint (34,651,227 bp monomer 651) and then by how close their midpoint is to Sox2. With `prefer_flanking = True`, flanking pairs always beat non-flanking pairs even if the non-flanking midpoint is closer.
3. **Convergent orientation** (`require_convergent = True`). Keep only pairs whose left site is + (forward) and whose right site is - (reverse) — the textbook signature of a stalled-cohesin loop anchor.

Size window: `min_sep_monomers = 200` (200 kb at 1 kb/monomer; below this, MSD is dominated by local relaxation rather than the loop), `max_sep_monomers = 1500` (sanity upper bound).

Picks `n_pairs = 2` non-overlapping pairs (`disjoint_anchors = True`) so two pairs don’t share an endpoint.

What this resolves to with the current BEDs (run `python -c 'from configs.parameters import get_msd_pairs; print(get_msd_pairs("mESC"))'` to reproduce):

cell type	pair (monomer, monomer)	genomic separation	flanks Sox2
mESC	(601, 1103)	502 kb	yes
mESC	(647, 1323)	676 kb	yes
neuron	(601, 1103)	502 kb	yes
neuron	(647, 1323)	676 kb	yes

The two cell types pick the **same pairs** (same monomer indices), which is what makes the cross-condition MSD overlay meaningful.

If nothing satisfies the rules, `relax_if_missing = True` first drops conservation, then drops convergence; if even that empties the candidate list, `_auto_select_msd_pairs` raises with a mes-

sage that tells you to set `pairs_mesc` / `pairs_neuron` explicitly.

Override. Set `pairs_mesc = [(a, b), (c, d), ...]` and (optionally) `pairs_neuron` in `MSD_PROBE` to supply pairs directly. The list can be any monomer indices; the analysis just replays them without validation. Use this when you want to test a specific hypothesis (e.g. a pair that is mESC-only). For a conservation-preserving cross-type comparison leave both lists as `None` and let the auto-selector drive.

polymer_dynamics.py. Diagnostics that complement MSD: radius of gyration over time (`compute_rg_timecourse`), looped-fraction scalar per pair (`compute_looped_fraction`), dwell-time distribution in the contact state (`compute_dwell_times`), pair-separation densities (`compute_pair_distance_distribution`).

6h. analysis/calibration.py

Maps simulation units to physical units. Two anchor options:

- `hic`: align simulation $P(s)$ to experimental $P(s)$ in log-log, read off nm/monomer from the horizontal shift; read off sec/frame from how many frames reach the Hi-C equilibrium plateau.
- `msd`: requires an experimental MSD CSV (from live-cell imaging); match simulated $MSD(t)$ to experimental $MSD(t_{real})$.

Outputs a `calibration.json` dataclass dump with every anchor constant.

6i. analysis/batch_analysis.py, plot_results.py, lef_lifetimes.py, msd_statistics.py

Helpers used elsewhere in the pipeline:

- `batch_analysis.py` — standalone script that scans `results/polychrom_3d/`, runs contact-map analysis on every completed directory, and produces a single multi-panel comparison figure. Older and simpler than `run_analysis_all.py`; use it for a quick visual pass on a fresh batch.
- `plot_results.py` — generates the cross-condition overlay figures ($P(s)$, insulation, APA) called from `cluster/submit_plots.sh`.
- `lef_lifetimes.py` — statistics on LEF residence time distribution from the LEF-only trajectory files.
- `msd_statistics.py` — bootstraps MSD curves across replicates.

6j. scripts/summarize_analysis.py

Purpose. Walk every condition's pooled analysis files, collect scalar metrics ($P(s)$ slope, APA enrichment, Pearson with Hi-C, MSD exponent, calibration nm/monomer...) into one CSV / XLSX table.

How to run it.

```
python scripts/summarize_analysis.py \  
    --analysis-dir results/analysis \  
    --reference-condition mESC_ctrl
```

Outputs. Under `--analysis-dir`:

File	Contents
<code>summary_analysis_table.csv / .xlsx / .json</code>	One row per condition $\times$ replicate, one column per metric.
<code>summary_stats_all_pairs.csv</code>	Pairwise bootstrap tests between every pair of conditions.
<code>summary_stats_vs_reference.csv</code>	Each condition vs the reference (default: first one discovered, or <code>--reference-condition</code>).

6k. `scripts/check_results.py`

Quick inventory script. Lists every directory under `--results-dir` (default `results/polychrom_3d`), how many frames it has, whether the analysis sub-directory is populated. Nothing fancy, meant to be run after a big SLURM job to confirm nothing fell off the cluster.

Cluster submit scripts

Every script in `cluster/` is a SLURM wrapper. They all share the same environment setup (`mambaforge` $\rightarrow$ `conda activate polychrom`) and `cd "${SLURM_SUBMIT_DIR}"` so you can `sbatch` them from the repo root. Most also create the `logs/` directory if missing.

A small number of scripts are **legacy** — superseded by newer wrappers but kept for one-off reruns, debugging, or backwards compatibility with old recipes you might find in commit messages and side projects. They are flagged below; do not delete them, but reach for the **CURRENT** wrappers when starting fresh.

The condition names in the SLURM `CONDITIONS=(...)` arrays must match `SIMULATION_CONDITIONS` in `configs/parameters.py`. The legacy `submit_analysis.sh` once had its array out of sync — fixed in commit 80929a8 — but the others have always been correct.

One-command launcher

`cluster/submit_pipeline.sh` (CURRENT) is the recommended entry point for a fresh sweep. It submits the simulation $\rightarrow$ merge $\rightarrow$ analysis chain as a single dependency tree, and the analysis stage is itself a chain of resume-capable jobs linked by `--dependency=afterany`. So a wall-time hit, an OOM, or a transient cluster fault on the analysis stage doesn't break the pipeline — the next link wakes up automatically and reuses whatever the previous link already produced (`--reuse-heavy`, `--skip-existing`).

```

bash cluster/submit_pipeline.sh                # full chain, 3 resume links
bash cluster/submit_pipeline.sh --skip-sim      # pick up at merge
bash cluster/submit_pipeline.sh --skip-sim --skip-merge # analysis only
bash cluster/submit_pipeline.sh --resume-links 5 # 5-link analysis chain
bash cluster/submit_pipeline.sh \
    --analysis-script cluster/submit_analysis_all_16cpu.sh # RAM-tight variant
bash cluster/submit_pipeline.sh --skip-sim --after 9457828 # attach to existing JID

```

The launcher prints the JID of every submission so you can `tail -f logs/...` or `scontrol show job` them.

Hi-C reference data preparation

Run these once if you ever need to rebuild `data/mcool/*.mcool` from raw Bonev 2017 sequencing data. They are **CURRENT** but rarely used in day-to-day work — the mcool files persist between sweeps.

- `cluster/submit_hic_pipeline.sh` — chains the full Option B pipeline (align → mcool → extract). Use for paper-quality from-scratch Hi-C.
- `cluster/submit_hic_mcool.sh` — just the `02_make_mcool.sh` step (cooler-build).
- `cluster/submit_hic_extract.sh` — just `03_extract_sox2.py` (locus extraction from a pre-computed cooler).
- `scripts/sbatch_bonev_hic.sbatch` — the fast Option A wrapper for the Sox2 chr3 slice; submit with `--export=CELLTYPE=ES|CN`.

Simulations (Tier 1 + Tier 2)

Tier 1 — 1D LEF dynamics

- `cluster/submit_lef_sweep.sh` — **CURRENT**. CPU array, 8 conditions × 3 reps = 24 jobs, ~10–30 min each. Runs `scripts/run_simulation.py --engine lef_only`. The 1D LEF trajectory is the input to Tier 2; this stage is required.

Tier 2 — 3D polychrom

- `cluster/submit_polychrom_multigpu.sh` — **CURRENT**. The canonical 3D entry point. 4-GPU array job: each array task takes one condition, splits the total blocks across 4 V100s, runs them in parallel as shards, then exits. The shard outputs land in `results/polychrom_3d/<cond>_rep<N>_shard<M>/` and require a subsequent `submit_merge_shards.sh` step. Reps start at offset 3 to avoid clashing with the legacy single-GPU reps 0–2 (which is what made keeping `submit_polychrom.sh` around useful).
- `cluster/submit_polychrom.sh` — **LEGACY**. Pre-shard single-GPU array (24 jobs, ~6–12 h each). Useful as a **fallback** when the multi-GPU `cuda` partition is unavailable, and to populate reps 0–2 alongside the multi-GPU reps 3+. Output goes directly to `results/polychrom_3d/<cond>_rep<N>/` (no `_shard<M>` suffix), so it bypasses the merge step. Don't reach for this unless you specifically need the no-shard layout.
- `cluster/submit_single_condition.sh` — **LEGACY**. Runs ONE condition on 4× V100; takes the condition name as the first positional arg. Useful for re-running a single condition that needs more reps or a different parameter, without launching the full array. Pre-shard era; still works on the current codebase.
- `cluster/submit_all_4gpu.sh` — **LEGACY**. Pre-shard top-level orchestrator that chained Tier 2 simulations + analysis as a single dependency tree (precursor to `submit_pipeline.sh`). Kept for documentation continuity; superseded by `submit_pipeline.sh`.

Merge

- `cluster/submit_merge_shards.sh` — **CURRENT**. Walks `results/polychrom_3d/`, finds every group of `<base>_rep<N>_shard<M>/` directories, and merges them into one `results/polychrom_3d/merged_<base>_rep<N>/`. 48 h walltime. Smart-skips groups

already merged (shard-count-based — see “File-output gotchas” in CLAUDE.md). Required step between Tier 2 and analysis.

- **cluster/submit_migrate_legacy.sh** — **ONE-TIME**. Migrates the pre-2026-04-27 layout (per-sim-dir <sim_dir>/analysis/ outputs) to the flat **results/analysis/** layout. Supports **DRY_RUN=1**, **COPY=1** (copy instead of move), **NO_RENAME=1** (don’t rename source dirs to merged_<base>). Idempotent on already-migrated trees. Once your tree is fully on the flat layout, this script idles — keep it for paper-trail reproducibility.

Analysis

- **cluster/submit_analysis_postmerge.sh** — **CURRENT** (preferred). Refuses to start if no merged_*/ dirs exist (so you don’t waste cluster time on an empty input). Calls **run_analysis_all.py** with all phases enabled (contact maps, P(s) + power-law, APA + AbLE, CTCF overlay, MSD, polymer dynamics, pooling, summary table). 32-CPU 2× parallel per dir, 24 h walltime, 360 GB RAM. Runs **plot_contact_map_panels.py** inline at the end. Picks up the current **submit_pipeline.sh** resume convention via **RESUME=1** and **SKIP_EXISTING=1**.
- **cluster/submit_analysis_all_32cpu.sh** — **CURRENT**. Older wrapper that does the same work as **submit_analysis_postmerge.sh** minus the merged-dir sanity check. Functionally equivalent; kept while older docs and muscle memory migrate over to the postmerge wrapper. New scripts should call the postmerge variant; existing recipes pointing at this one continue to work.
- **cluster/submit_analysis_all_16cpu.sh** — **CURRENT**. Sequential per-dir variant (16 cores, ~17 h for 17 conditions). Use when the cluster queue is full (smaller allocation, faster scheduling) or when memory pressure is high. Same env-var toggles as the 32-CPU variant.
- **cluster/submit_analysis_resume_chain.sh** — **CURRENT**. Standalone analysis-only resume chain: submits N analysis jobs linked by **--dependency=afterany**, all but the first run with **RESUME=1** and **SKIP_EXISTING=1**. Use when you only need the analysis stage chained (sim + merge already done) and the single 24/48 h walltime isn’t enough. **submit_pipeline.sh** reuses this pattern internally for its analysis stage.
- **cluster/submit_analysis.sh** — **LEGACY**. Pre-flat-folder, per-<cond>_rep<N> analysis array. Runs **analysis/contact_maps.py** on one directory per SLURM array task. Useful for re-running one bad replicate in isolation without touching the others. Output goes to <sim_dir>/analysis/, which the new pipeline ignores — see “File-output gotchas” / <sim_dir>/analysis/ is dead.
- **cluster/submit_analysis_cpu.sh** — **LEGACY**. Pre-flat-folder contact-map-only extractor. Superseded by the **submit_analysis_all_*.sh** and **submit_analysis_postmerge.sh** wrappers, which call the orchestrator and produce the full per-condition fingerprint instead of just the contact map.

Plots

- **cluster/submit_plot_panels.sh** — **CURRENT**. Standalone re-render of **results/figures/** panels from the existing **results/analysis/** outputs. Cheap (~30 min). Already invoked at

the end of every analysis run; use this when only the figures need refreshing (e.g. you want to tweak vmin/vmax in `plot_contact_map_panels.py` without re-running analysis).

- **cluster/submit_plots.sh** — **LEGACY**. Pre-shard cross-condition overlay plotter, calls `analysis/plot_results.py`. Superseded by the inline figure-panel rendering in the analysis wrappers. Kept so links from old session notes don't 404.

Helpers

- **cluster/check_partitions.sh** — **UTILITY**. Prints the partitions, QOS limits, and GPU availability you have access to. Run this first if you're on a new cluster or after a SLURM upgrade and don't know which queue to target. Read-only diagnostic.

Dependency-chain recipes

The single-command launcher (`submit_pipeline.sh`) covers the common case. The recipes below are for one-off chains where the launcher's defaults don't quite fit.

```
# Two-step chain: merge -> analysis
```

```
J_MERGE=$(sbatch --parsable cluster/submit_merge_shards.sh)
```

```
J_ANA=$(sbatch --parsable --dependency=afterok:${J_MERGE} \
    cluster/submit_analysis_postmerge.sh)
```

```
echo "Merge: ${J_MERGE}    Analysis: ${J_ANA}"
```

```
# Three-step chain (full sweep, manual): sim -> merge -> analysis
```

```
J_SIM=$(sbatch --parsable cluster/submit_polychrom_multigpu.sh)
```

```
J_MERGE=$(sbatch --parsable --dependency=afterok:${J_SIM}    cluster/submit_merge_shards.sh)
```

```
J_ANA=$(sbatch --parsable --dependency=afterok:${J_MERGE}    cluster/submit_analysis_postmerge.sh)
```

```
# Hi-C path (rebuild reference mcool, then sim+analysis):
```

```
J_HIC=$(sbatch --parsable cluster/submit_hic_pipeline.sh)
```

```
J_SIM=$(sbatch --parsable --dependency=afterok:${J_HIC} cluster/submit_polychrom_multigpu.sh)
```

```
J_MERGE=$(sbatch --parsable --dependency=afterok:${J_SIM}    cluster/submit_merge_shards.sh)
```

```
J_ANA=$(sbatch --parsable --dependency=afterok:${J_MERGE}    cluster/submit_analysis_postmerge.sh)
```

```
# Resume-chain on the analysis stage only (sim + merge already done):
```

```
bash cluster/submit_analysis_resume_chain.sh 5    # 5 chained links
```

```
bash cluster/submit_analysis_resume_chain.sh 3 9457828    # attach to JID 9457828
```

```
# Legacy multi-step recipe (kept for backwards compatibility with old
```

```
# session notes - prefer submit_pipeline.sh for new work):
```

```
JOB_ES=$(sbatch --export=CELLTYPE=ES scripts/sbatch_bonev_hic.sbatch | awk '{print $NF}')
```

```
JOB_CN=$(sbatch --export=CELLTYPE=CN scripts/sbatch_bonev_hic.sbatch | awk '{print $NF}')
```

```
JOB_EX=$(sbatch --dependency=afterok:${JOB_ES}:${JOB_CN} \
    cluster/submit_hic_extract.sh | awk '{print $NF}')
```

```
JOB_LEF=$(sbatch cluster/submit_lef_sweep.sh | awk '{print $NF}')
```

```
JOB_SIM=$(sbatch cluster/submit_polychrom.sh | awk '{print $NF}')    # legacy single-GPU
```

```
JOB_ANA=$(sbatch --dependency=afterok:${JOB_EX}:${JOB_SIM} \
    cluster/submit_analysis_cpu.sh | awk '{print $NF}')    # legacy
```

```
sbatch --dependency=afterok:${JOB_ANA} cluster/submit_plots.sh # legacy
```

afterok vs afterany — which one to use

- `--dependency=afterok:<JID>` — fire only if the upstream job finished with exit-status 0 (COMPLETED in SLURM-speak). Use this between pipeline stages where the next stage is meaningless if the previous one failed (sim -> merge, merge -> analysis-cold-link).
- `--dependency=afterany:<JID>` — fire regardless of how the upstream job ended (COMPLETED / FAILED / TIMEOUT / CANCELLED). Use this for resume-chain links where the *whole point* is to recover from wall-time TIMEOUTs. `submit_pipeline.sh` and `submit_analysis_resume_chain.sh` use `afterany` between analysis links for exactly this reason.
- Stop a chain by `scancel`ing every link (afterany ignores cancellation of the upstream job, so cancelling just the first link doesn't stop the chain).

Conventions you'll see everywhere

- **Monomer = 1 kb.** One bead in the polymer corresponds to 1,000 bp of real DNA. So monomer index `i` covers genomic coordinates `REGION_START + i * RESOLUTION` to `REGION_START + (i+1) * RESOLUTION`.
- **Tiled chromosome.** Simulations actually run on a chain of length `TILING["chrom_size"]` (~70,000 monomers) = 28 tiled copies of the 2 Mb locus with padding. Contact maps are collapsed back down to `N_MONOMERS` (2,000) for comparison. The tiling is an error-bar trick — ~28× more data per wall-clock second without perturbing the physics of any one locus.
- **Output directory names.** Always `<params_name>_ctcf-<ctcf_type>_rep<N>` for merged directories, `<params_name>_ctcf-<ctcf_type>_rep<N>_shard<S>` for shards. The `_ctcf-<ctcf_type>_` suffix is the single most important convention: it keeps mESC-CTCF and neuron-CTCF runs of the same cohesin parameter set separated on disk.
- **RNG seed.** `42 + replicate * 1000` for one-shot runs, `42 + replicate * 10000 + shard * 1000` for shards. Always derived from the replicate / shard index, so reruns are reproducible.
- **Every run writes a `params.json`** with the full parameter dict + cell type + tiling + replicate. If a result file ever looks wrong, check `params.json` first — it is the single source of truth for what actually ran.
- **Condition name vs params name.** `SIMULATION_CONDITIONS` entries have a `name` (e.g. `CN_long_residency_neuron_ctcf`) that is what you pass to `--condition`, and a `params` dict whose `name` (e.g. `CN_long_residency`) is what the output directory starts with. The two differ because the same cohesin params can be paired with either CTCF track.

Common pitfalls

“No results found at ...” from `submit_analysis.sh`. The legacy single-directory analysis submit script asks `configs.parameters` to resolve the condition name into (`params_name`, `ctcf_type`) and then constructs `<params_name>_ctcf-<ctcf_type>_rep<N>`. If the directory doesn't exist, it skips silently. Check that the sim finished (`ls results/polychrom_3d/<params_name>_ctcf-<ctcf_type>_rep<N>`) — if not, the simulation crashed earlier in the pipeline.

ModuleNotFoundError: configs or analysis. Every script in `scripts/` inserts the repo root

onto `sys.path`. If you moved a script somewhere else, or sourced it in an unusual way, this breaks. Fix: run from the repo root, or add the repo root to `PYTHONPATH`.

KeyError: 'bin-size' in Hi-C prep. Old cooler; see §2a note. Solution: use `scripts/download_bonev_hic.sh` which avoids `cooler makebins` entirely.

Wrong Hi-C reference compared to the simulation. The routing in `run_analysis_all.py::HIC_CONDITION` infers cell type from the directory name. If a directory has an unusual name, the fallback rule picks mESC; a warning is logged but the run continues with the wrong reference. Either rename the directory to match the `ctcf-<type>` convention or add a new regex rule at the top of `HIC_CONDITION_MAP`.

Multi-GPU shards where only some succeeded. The merge step (`merge_shards.py`) counts present shard directories. If only 2 of 4 merged, find which shard failed in `logs/*_shard<i>.err`, resubmit with `--shard-index-offset` to avoid re-running the successful shards, then re-merge. `merge_metadata.json` inside the merged directory records which shards contributed.

A run finished but `analysis/sim_contact_map.npy` is missing. Check the end of the SLURM log for an OOM (Killed signal) or import error. Re-run with `--skip-existing` so already-processed directories aren't redone.